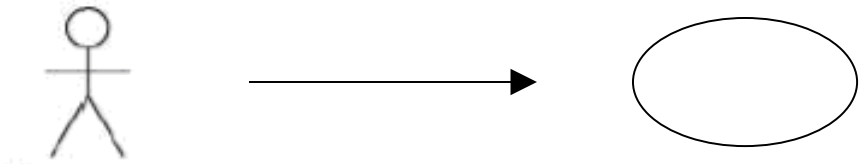
System Analysis & Design

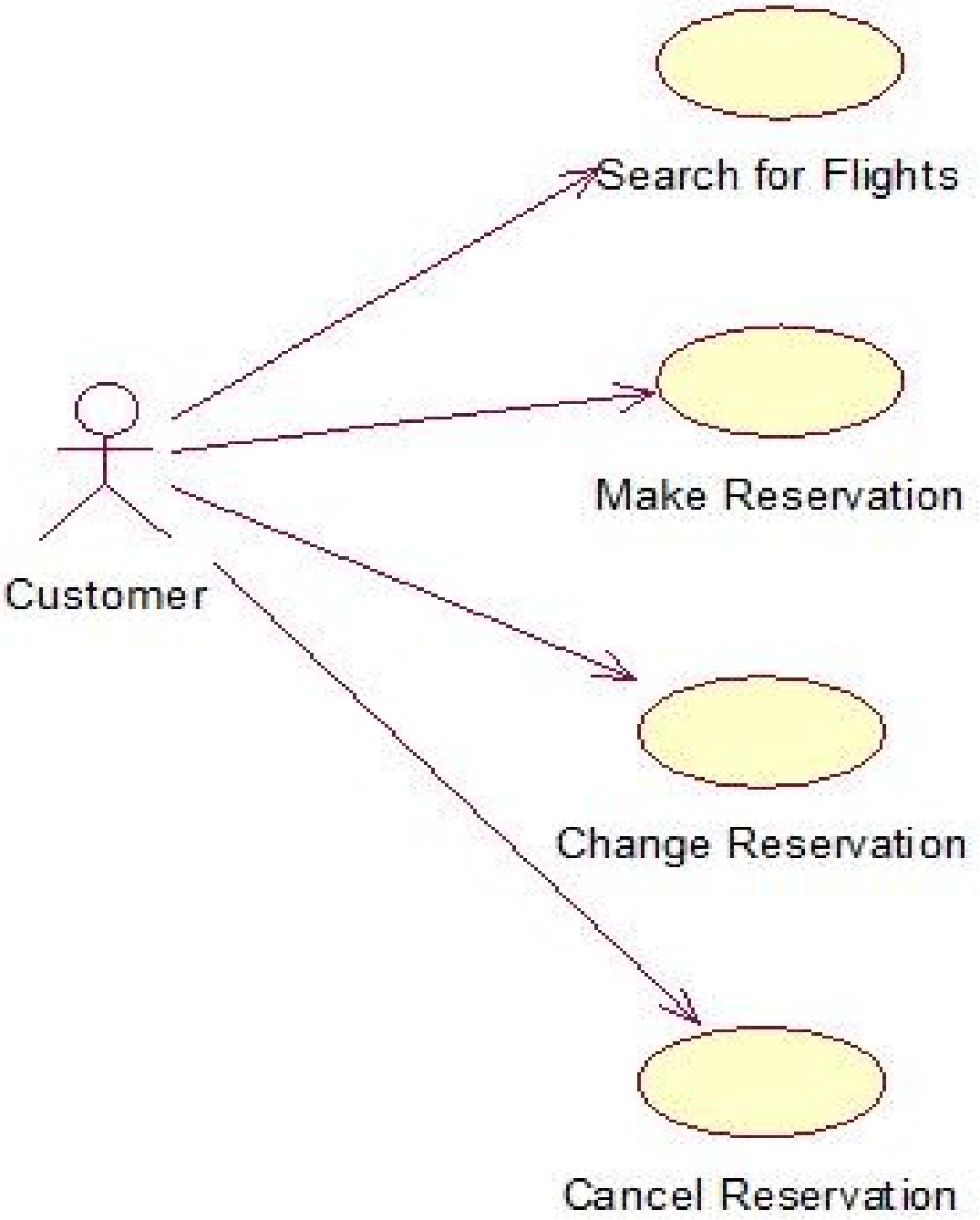
Use Case Diagram

Use Case Diagram

- Use cases represent system functionality, the requirements of the system from the *user's perspective*
- Use cases just focus on *automated processes*
- Use case diagrams show the *interactions between*
  - *Use cases:* what the system should do, and
  - *Actors:* anyone or anything that interacts with the system (Individual, group, company,...)



Example


Search for Flights
Make Reservation
Customer
Change Reservation
Cancel Reservation

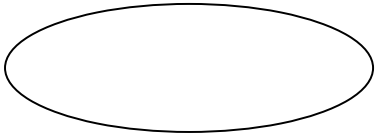
Actor



- An actor is anyone or anything that is *outside* the system's scope but *interacts* with it (Individual, group, company,...)

- There are three types of actors:
  - Users of the system
    - physical person, or a user who will be *directly* using the system
  - Other systems that will interact with the system being built
  - Time
    - Time becomes an actor when the passing of a certain amount of time triggers some event in the system *(out of control)*

Use Case



- It is the functionality the system will provide a value to the end user

- Use cases are an *implementation-independent:*
  - *High-level* view of what the user expects from the system
  - Focus on *what* the system should do, *not how* the system will do it

- A typical system will have somewhere between *20 and 70* use cases

- The use cases should be named in *user terms*, *not technical terms*, and should be *meaningful to the customer*

Use Case *cont*

- *So, when you have the final list of use cases, how do you know if you've found them all?*

- Is each functional requirement in at least one use case? If a requirement is not in a use case, it will not be implemented.

- Have you considered how each end user will be using the system?

- What information will each end user be providing for the system?

- What information will each end user be receiving from the system?

- Have you considered maintenance issues? Someone will need to start the system and shut it down.

- Have you identified all of the external systems with which the system will need to interact?

- What information will each external system be providing to the system and receiving from the system?

Use Case: *Flow of Events*

- To actually build the system, though, you'll need more *specific details*. These details are written as the flow of events

- The purpose of the flow of events is to *document the flow of logic through the use case*

- Although it is detailed, the flow of events is still *implementation-independent*

Use Case: *Flow of Events* *cont*

- This document will describe in detail *what the user of the system will do and what the system itself will do*

- Notice the *pattern* in the flow of events:
  - The user does something, then
  - The system does something in response,
  - Then the user does something, then the system responds, and so on

Use Case: *Flow of Events* cont

It includes:

- *A brief description: Each use case should include a *short description* that explains what the use case will do

- *Preconditions:* list any conditions that have to be met *before* the use case can start at all. For example, the precondition for one use case may be that another use case has run

- **Flow of events*

- *Post conditions:* are conditions that must always be *true* after the use case has finished executing. Like preconditions, post conditions can be used to add information about the *order* in which the use cases are run

Use Case: *Flow of Events Types*

- There are three types of flows:
  - *Primary flow* is the *"happy day"* scenario, or the most frequently used path through the use case
  - *Alternate flows* are deviations from the primary flow that do not suggest an error condition
  - *Error flows* are deviations from the primary or alternate flows that suggest some sort of *error condition*. Error flows suggest that there is a problem with the system itself

Use Case: *Flow of Events Users*

- There are three primary users of the flow of events:

  1- The *customers* will be reviewing this document to make sure it accurately reflects their expectations

  2- The *system designers* will be using it to create the system design and eventually to build the system

  3- The *quality assurance team* will use the flow of events to create test scripts

The flow of events must give them enough information to understand the *sequence* of events that needs to occur through the use case

Relationships

- The *association relationship* is used to show the relationship between a use case and an actor

- There are two types of relationships between use cases
  - *Includes* relationship
  - *Extends* relationship

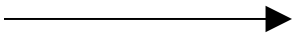
  These relationships are used when there is a *certain amount of commonality between the use cases*

- *Generalization relationship* is the only relationship allowed between actors

Relationships: *Association*

- *Association relationship* is used to show the relationship between a use case and an actor

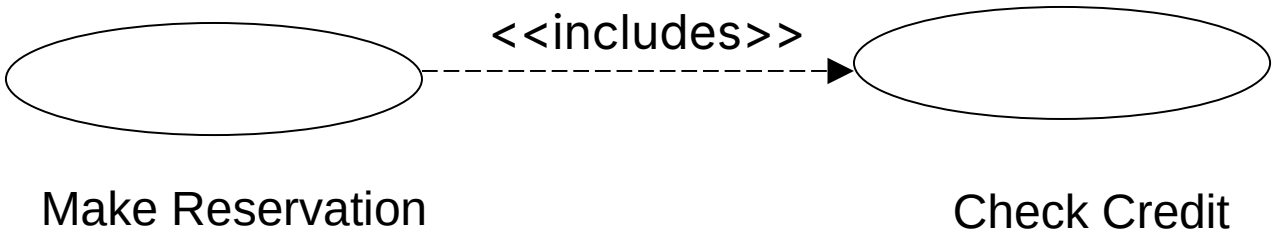
- Every use case *must be initiated* by an actor, With the *exception* of use cases in includes and extends relationships



Relationships: *Includes*

- *Includes relationship* allows one use case to use the functionality provided by another use case
- This relationship can be used in one of two cases:
  - **First**, if two or more use cases have a large piece of functionality that is identical
  - **The second case** where an includes relationship is helpful is a situation in which a single use case has an unusually large amount of functionality

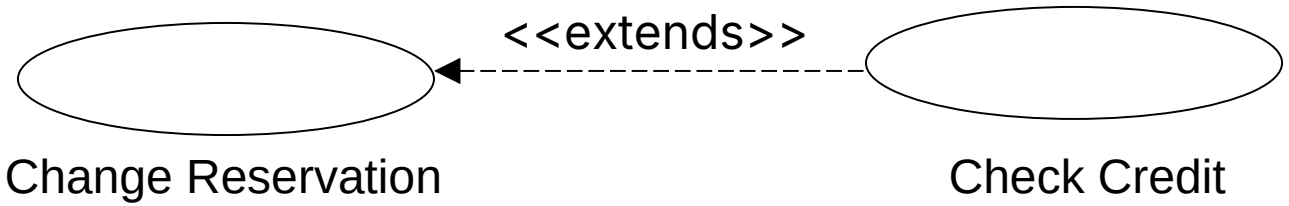
- An includes relationship suggests that one use case *always* uses the functionality provided by another



Relationships: *Extends*

- *Extends relationship* allows one use case the *option* to extend the functionality provided by another use case

- It is very similar to an includes relationship, because in both of these types of relationships, you separate some common functionality into its own use case



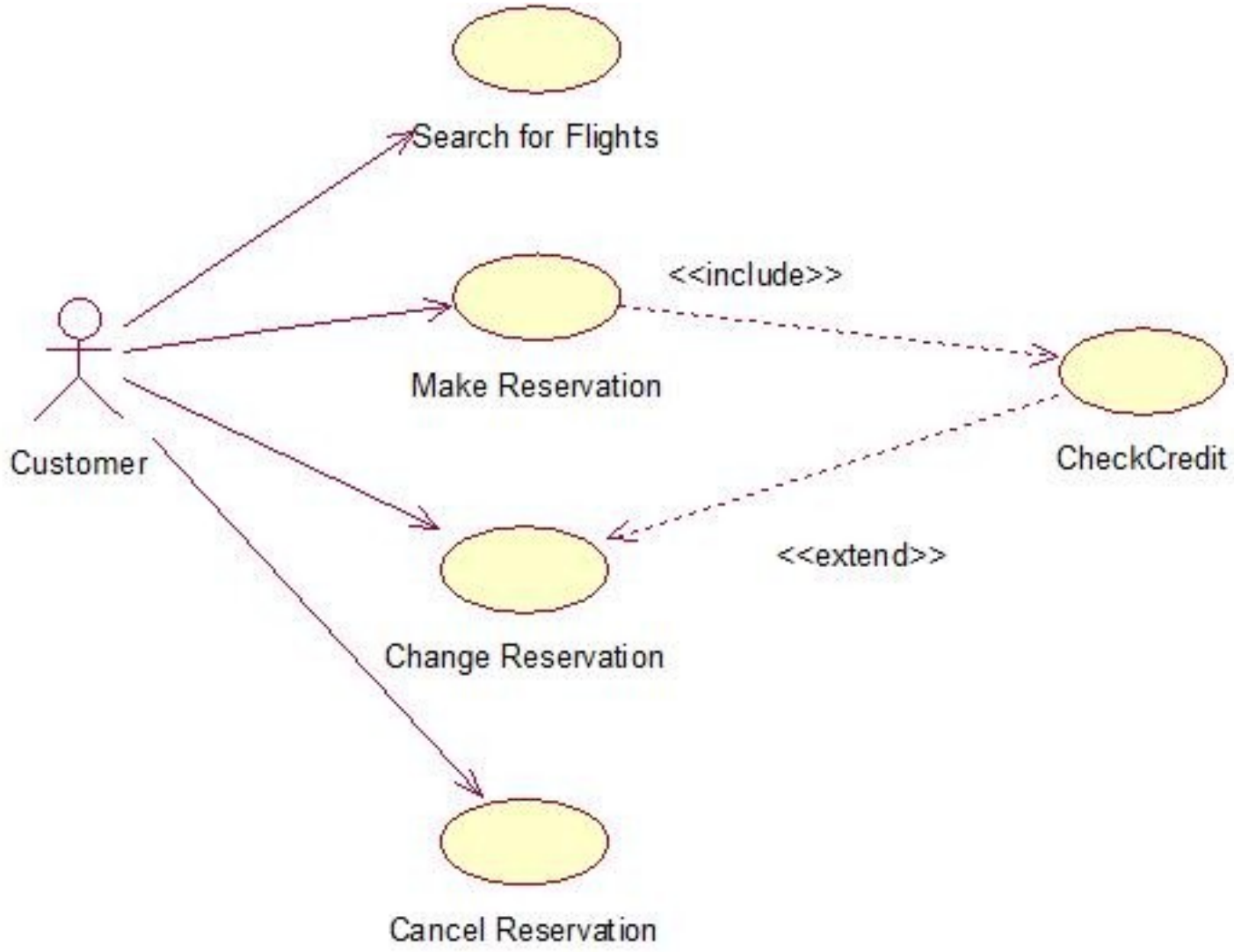
Abstract Use Case

- An *abstract use case* is one that is *not started directly by an actor*

- Instead, an abstract use case provides some additional functionality that can be used by other use cases

- Abstract use cases are the use cases that participate in an *includes or extends relationship*

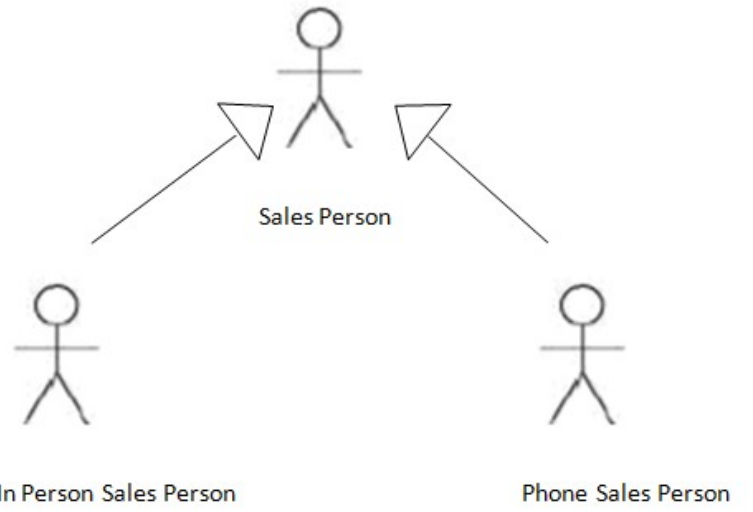
Example


Search for Flights
<<include>>
Make Reservation
Customer
CheckCredit
<<extend>>
Change Reservation
Cancel Reservation

Relationships: *Generalization*

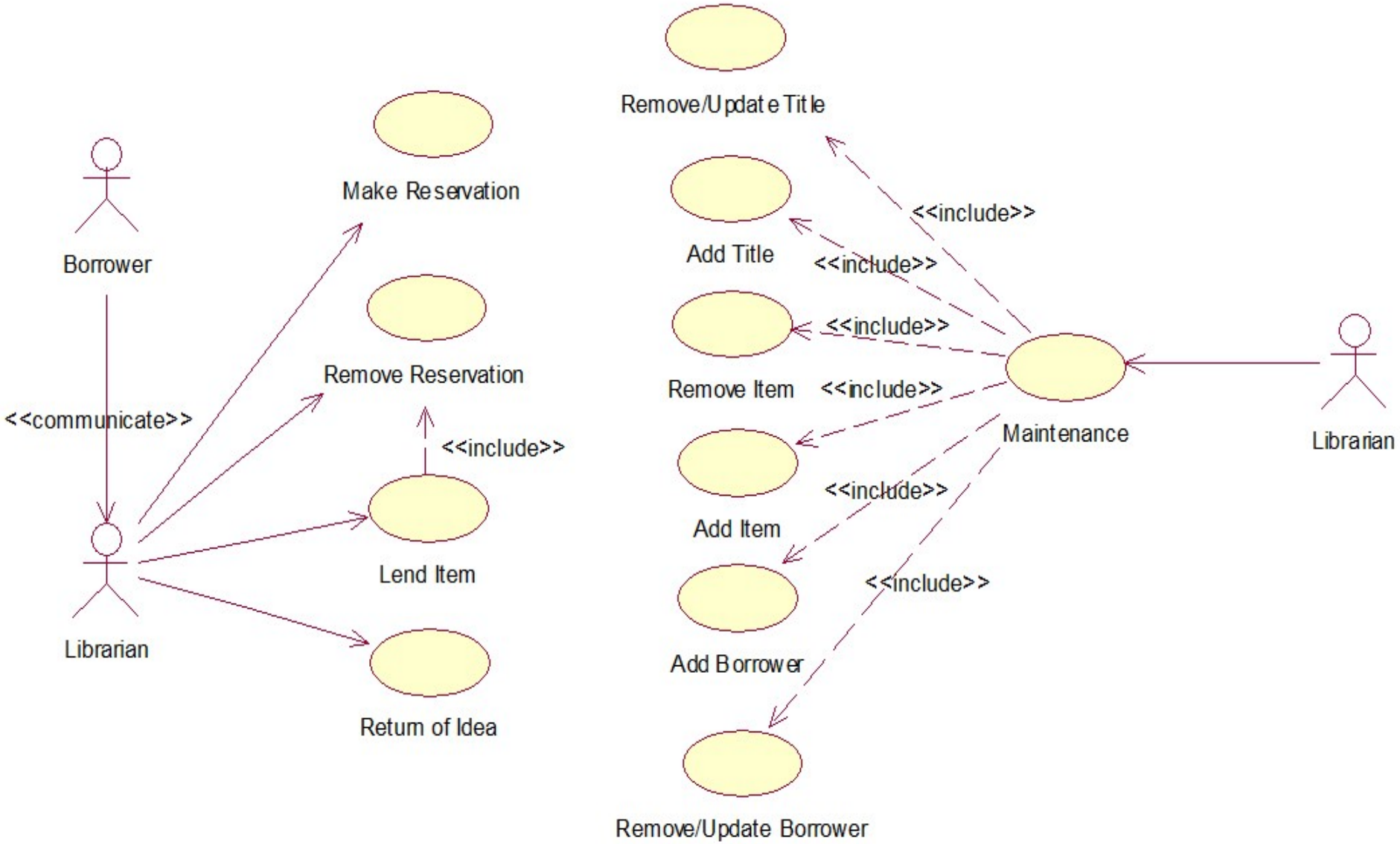
- *Generalization relationship* is used to show that several actors

- For example, you may have two types of customers. If the type A customers will be initiating some use cases that type B customers will not, it's probably worth including the actor generalizations. If both types of customers use the same use cases, it's probably not necessary to show an actor gen



Practice

- Draw a Use Case Diagram shows the interactions between the use cases and actors of a Library Managemnt system


Borrower
Make Reservation
Remove Reservation
<<communicate>>
<<include>>
Lend Item
Librarian
Return of Idea
Remove/Update Title
Add Title
<<include>>
<<include>>
Remove Item
<<include>>
Maintenance
Librarian
Add Item
<<include>>
<<include>>
Add Borrower
Remove/Update Borrower

Thank You ☺